

ATHENA AI

Enterprise Multi-Agent Platform & User Manual

Author	Athena AI Core Engineering
Status	Deployed & Operational (Local Stack)
Gateway Port	http://localhost:8080
Version	v2.0 Enterprise SaaS

1. Product Overview

Athena AI is a production-grade, containerized Multi-Agent Automation Platform designed for enterprise operations. Built using FastAPI, LangGraph, React, Celery, and Redis, the platform facilitates complex task automation through collaborative specialized agents overseen by a centralized supervisor agent.

Key Core Capabilities:

- **Hierarchical Multi-Agent Collaboration:** LangGraph orchestration distributes tasks from a central Supervisor to dedicated agents (SQL, Document, Research, Code, and Workflow).
- **AI Memory Vault (RAG):** Vector-based semantic database leveraging ChromaDB for high-accuracy contextual retrieval and metadata-isolated organization.
- **Distributed Asynchronous Workers:** Multi-stage operations like OCR, parsing, and analytical tasks run asynchronously in Celery background workers.
- **Observability Suite:** Unified system metrics exposed via Prometheus and visually dashboarded using Grafana out-of-the-box.

2. System Architecture & Components

The application is structured as a collection of microservices defined in a single orchestrating Compose stack:

Service	Image/Tech	Function
API Gateway	Nginx (Port 8080)	Single entry point routing requests to Frontend/Backend.
Frontend UI	React / Vite	Single Page App serving chat console, admin dashboard.
Backend API	FastAPI (Python)	Processes core business logic, LangGraph runs, agent sessions.
Celery Worker	Python / Celery	Performs async tasks, background processing, document parsing.
Redis Broker	Redis Alpine (Port 6380)	In-memory broker, rate-limiter, cache database.
Prometheus	Prometheus (Port 9091)	Gathers and aggregates system logs, speed metrics.
Grafana	Grafana (Port 3002)	Telemetry dashboards for monitoring load and latency.

3. Step-by-Step Usage Guide

Step 3.1: Accessing the Portal

Open your web browser and navigate to **http://localhost:8080**. You will be greeted with the premium Athena AI dashboard landing page. The gateway routes requests seamlessly behind the scenes.

Step 3.2: User Authentication & Role Setup

1. Click on the **Register** modal on the landing screen to create a new user profile.
2. Provide your name, email, credentials, and choose your **Department** (e.g., Engineering, HR, Legal).
3. Login to get your session JWT tokens automatically stored for authorization.

Step 3.3: Chat Console & Multi-Agent Engagement

Once logged in, open the **Chat Interface** page:

- Ask complex prompts: e.g., *'Run research on recent database transactions and write Python to clean it.'*
- The system initializes the **Supervisor Agent** which decomposes your request, elects the appropriate specialized agent (e.g., Code Agent, Research Agent), and stream tokens using Server-Sent Events (SSE) back to the UI in real time.

Step 3.4: RAG Ingestion (Memory Vault)

To query external data:

1. Navigate to the **Document Vault** tab.
2. Upload any enterprise document (e.g., PDFs, CSVs).
3. The background Celery worker parses the document, segments it into chunks, embeds it using vectors, and indexes it within **ChromaDB**.
4. Future queries automatically reference these vectorized context sources.

4. System Health & Observability Dashboards

Athena AI exposes system logs, memory statistics, and process queue metrics via the monitoring services:

Grafana Visualizations (Port 3002)

Grafana maps and charts metrics recorded by Prometheus. Log in with admin credentials:

- **URL:** *http://localhost:3002* (Admin Credentials: admin / admin)
- Check API Latency trends, queue bottlenecks, active Celery worker distribution, and container memory footprints.

Prometheus Metrics scraping (Port 9091)

Directly query metric variables or verify active scrapers:

- **URL:** *http://localhost:9091*

API interactive Documentation Swagger

If you are developing integrations with Athena AI or need to see the REST endpoints directly:

- **URL:** *http://localhost:8080/api/docs*

5. Operations Quick Reference

Common administration commands inside the repository directory:

```
Start Cluster:  
docker compose up -d  
  
Rebuild Containers:  
docker compose up -d --build  
  
Stop and Clean Stack:  
docker compose down --remove-orphans  
  
Retrieve Container Logs:  
docker compose logs -f backend
```